



PLAN DE PRUEBAS

Test Plan

Proyecto: LibroClaro

Plataforma web de libros de texto SEP con correcciones verificadas, material complementario y suscripciones simuladas

Stack: TypeScript · React + Vite · Express · Prisma + PostgreSQL · Mongoose + MongoDB · Docker

Equipo

Mariano Jara · Carlos Morrison · Stella Casillas · Luis Fernando Maldonado

Materia: Calidad de Software

Fecha de emisión: 21 de junio de 2026

Control del documento

Este Plan de Pruebas es un documento vivo, versionado en el repositorio del proyecto bajo /docs/qa/test-plan.docx. Cualquier modificación debe registrarse en la siguiente tabla y ser aprobada por el QA Lead antes de su entrada en vigor.

Versión	Fecha	Autor	Cambios
1.0	2026-06-20	Stella Casillas (QA Lead)	Versión inicial aprobada

Rol	Responsable	Aprobación
Elaboró	Stella Casillas - QA Lead	Firma electrónica
Revisó	Luis Fernando Maldonado - QA Automation	Firma electrónica
Aprobó	Product Owner	Firma electrónica

1. Introducción y objetivos

1.1 Propósito del documento

Este Plan de Pruebas define el alcance, enfoque, recursos, calendario y criterios bajo los cuales se ejecutarán las actividades de aseguramiento de calidad del proyecto LibroClaro. Sirve como acuerdo formal entre los equipos de desarrollo, QA y Producto sobre qué se prueba, cómo, cuándo y bajo qué condiciones el producto se considera apto para liberarse a producción.

1.2 Descripción del producto

LibroClaro es una plataforma web que ofrece libros de texto de la SEP corregidos y verificados por expertos. Los errores en las páginas aparecen resaltados y, al pasar el cursor por encima, se muestra una corrección con citas a fuentes confiables. La plataforma también ofrece material complementario por página y un modelo de suscripción freemium (Gratis, Pro, Institucional). El sistema se compone de cuatro capas claramente diferenciadas:

- **Frontend Web:** aplicación React 18 + Vite + TypeScript con MUI v6, TanStack Query, axios, React Router y react-pdf para renderizar los libros.
- **API REST:** Express + TypeScript con autenticación JWT (bcrypt), validación con Zod, Multer para subida de PDFs, pdf-lib + sharp + pdf2pic para generar portadas y PDFs anotados.
- **BD Relacional (PostgreSQL vía Prisma):** tablas User, Institution, Subject, GradeLevel, Book, Annotation, AnnotationView (para la cuota Free de 20/mes) e Invoice.
- **BD No Relacional (MongoDB vía Mongoose):** colección SupplementaryMaterial con el material complementario por página y por libro.

Existen tres tipos de usuario (Docente, Administrador de Institución, Editor) y tres planes de suscripción (Gratis \$0, Pro \$100/mes, Institucional \$2500/mes) con beneficios diferenciados.

1.3 Objetivos del plan

1. Verificar que el registro y autenticación funcionen correctamente, incluyendo la validación de edad mínima (≥ 18 años) y el manejo seguro de contraseñas con bcrypt.
2. Garantizar que el catálogo, los filtros (materia, grado, ciclo escolar) y el buscador devuelvan resultados correctos y consistentes.
3. Asegurar que el lector PDF, el hover de anotaciones y el descuento de cuota mensual del plan Gratis (20 vistas únicas/mes) funcionen sin errores.
4. Validar los flujos del editor: subir libro, dibujar anotaciones por coordenadas normalizadas, publicar/ocultar y gestionar editores.
5. Validar los flujos de Administrador de Institución: renombrar institución, crear/añadir/editar miembros, y el bloqueo correcto del perfil para docentes institucionales.

6. Validar el flujo de suscripciones simulado (Gratis → Pro → Institucional), la descarga del PDF con anotaciones integradas (Pro/Institucional) y el historial de facturas.
7. Alcanzar los niveles de cobertura objetivo definidos en la sección 12 (Métricas) y mantenerlos en cada release.
8. Detectar y prevenir vulnerabilidades alineadas con OWASP Top 10 mediante OWASP ZAP y revisión manual.
9. Establecer un proceso reproducible de gestión de defectos con trazabilidad completa requisito ↔ caso ↔ bug ↔ fix.

1.4 Audiencia

Este documento está dirigido a: equipo de QA (Mariano Jara, Carlos Morrison, Stella Casillas, Luis Fernando Maldonado), equipo de desarrollo, Product Owner y stakeholders del proyecto LibroClaro.

2. Alcance

2.1 Dentro del alcance (In scope)

Las siguientes funcionalidades y componentes serán objeto de pruebas formales:

- Autenticación: registro con validación ≥ 18 años, login con email/password, generación y validación de JWT, logout.
- Perfil de usuario: edición de datos, eliminación de cuenta (con bloqueo correcto para docentes institucionales).
- Catálogo de libros: listado en cuadrícula por grado, buscador y filtros por título, materia, grado y ciclo escolar.
- Detalle de libro: portada, metadatos, botón de abrir lector.
- Lector PDF: navegación entre páginas, render de áreas resaltadas (rojo = error, amarillo = error parcial), hover de anotaciones, panel lateral de material complementario.
- Cuota Free: descuento de la cuota mensual al revelar una anotación (GET /api/annotations/:id/reveal), tope de 20/mes, reseteo mensual.
- Descarga de PDF: original (todos los planes) y anotado (Pro/Institucional).
- Panel Editor: subida de PDF (multipart/form-data), publicar/ocultar libros, eliminar, gestión de editores, dibujo de anotaciones por arrastre, edición Markdown.
- Panel Institución: renombrar institución, crear/añadir/editar/eliminar miembros.
- Suscripciones: comparativa de planes, checkout simulado, historial de facturas.
- Endpoints REST: /auth, /users, /books, /annotations, /catalog, /editors, /institutions, /materials, /subscriptions.
- Persistencia: Prisma sobre PostgreSQL y Mongoose sobre MongoDB.
- Infraestructura local con Docker Compose (postgres + mongo + api + web).
- Compatibilidad cross-browser: Chrome, Firefox, Edge y Safari (últimas versiones).

2.2 Fuera del alcance (Out of scope)

- Integración con pasarelas de pago reales (el checkout es simulado por diseño).
- Localización a idiomas distintos del español.
- Auditorías formales de penetración por terceros certificados.
- Compatibilidad con navegadores legacy (IE 11, Safari <14).
- Pruebas de la infraestructura de DigitalOcean (entorno de hosting).
- Funcionalidades marcadas como experimentales o detrás de feature flags apagados.

2.3 Funcionalidades críticas (priorización)

Prio	Funcionalidad	Capas involucradas	Tipo de prueba mínima
P0	Login y registro con validación ≥ 18 años	Web + API + Postgres	Unit + E2E + Seguridad
P0	Lector PDF + hover de anotaciones	Web + API + Postgres	Unit + E2E
P0	Cuota Free 20/mes (annotations/reveal)	Web + API + Postgres	Unit + E2E + Performance
P0	Subida de libro PDF (Editor)	Web + API + Postgres + FS	Unit + E2E + Performance
P0	Descarga PDF anotado (Pro/Inst)	Web + API + FS	Unit + E2E + Performance
P1	Catálogo: filtros y buscador	Web + API + Postgres + Mongo	Unit + E2E
P1	Suscripciones y facturas	Web + API + Postgres	Unit + E2E + Seguridad
P1	Panel Institución (CRUD miembros)	Web + API + Postgres	Unit + E2E + Seguridad
P1	Editor de anotaciones (dibujo + Markdown)	Web + API + Postgres	Unit + E2E
P2	Material complementario por página	Web + API + Mongo	Unit + E2E

3. Estrategia de prueba por capa

La estrategia sigue la pirámide de pruebas: muchas pruebas unitarias (rápidas y baratas), pocas pruebas E2E (lentas pero de alto valor), pruebas de rendimiento sobre los endpoints críticos y pruebas de seguridad alineadas con OWASP Top 10. Las herramientas se asignan por tipo de prueba, no por capa: una misma herramienta puede cubrir varias capas.

3.1 Resumen de tipos de prueba por capa

Capa	Unit (Jest)	E2E (Cypress)	Performance / Seguridad
Frontend Web (React + Vite)	Componentes, hooks, utilidades y servicios HTTP de la web	Flujos completos de usuario en navegador real	ZAP escanea la web; JMeter mide la carga sobre la API que el front consume
API REST (Express + TS)	Controllers, services (auth, quota, pdf, markdownPdf, storage), middleware (auth, errorHandler), validadores Zod	Cypress llega vía la web, pero también ataca endpoints con cy.request en setup	ZAP escanea endpoints REST; JMeter ejecuta load, stress, spike y soak
BD Relacional (PostgreSQL + Prisma)	Repositorios mockeados en unitarias; queries reales con Prisma en pruebas de integración con BD de test	Cobertura indirecta vía E2E: confirmar que los datos quedan persistidos	JMeter detecta cuellos de botella (N+1, índices); ZAP intenta inyección
BD No Relacional (MongoDB + Mongoose)	Modelos y schema de SupplementaryMaterial validados con mongodb-memory-server	Cobertura indirecta vía E2E del material complementario en el lector	JMeter incluye lectura de material; ZAP verifica inyección NoSQL

3.2 Capa 1 - Frontend Web (React + Vite + TypeScript)

- **Unit (Jest + React Testing Library):** componentes presentacionales (BookCard, AnnotationTooltip, FilterBar, MarkdownPreview), hooks de TanStack Query, helpers de fecha (validación ≥ 18) y formato Markdown.
- **E2E (Cypress):** flujos completos en navegador real. Selectores con data-testid. Login vía cy.request en beforeEach.
- **Seguridad (OWASP ZAP):** baseline scan sobre la URL de staging del front; análisis pasivo y activo.

3.3 Capa 2 - API REST (Express + TypeScript)

- **Unit (Jest + Supertest):** controllers de auth, books, annotations, catalog, editors, institutions, materials, subscriptions y users; servicios (quota.service para la cuota Free, pdf.service y markdownPdf.service para la generación de PDFs anotados, auth.service para JWT y bcrypt, storage.service para Multer); middleware (auth, errorHandler, upload); validadores Zod de los DTOs de entrada.
- **E2E (Cypress):** los E2E recorren la API automáticamente al ejecutar los flujos web, pero además se usan cy.request para autenticar y preparar datos antes de cada test.

- **Performance (JMeter):** Load/Stress/Spike/Soak - ver sección 4.
- **Seguridad (OWASP ZAP + checklist OWASP Top 10):** Active scan en staging contra /api; revisión manual de A01, A02, A03, A05 y A07 (ver sección 3.6).

3.4 Capa 3 - BD Relacional (PostgreSQL + Prisma)

- **Pruebas con BD real:** contenedor PostgreSQL levantado con docker-compose para integración con Prisma. Migraciones (prisma migrate deploy) y seed ejecutados antes de la suite.
- **Aislamiento:** cada test hace truncate de las tablas modificadas en afterEach, o se ejecuta dentro de una transacción que se revierte.
- **Constraints verificados:** UNIQUE en User.email, UNIQUE en Institution.adminId, UNIQUE en Subject.name y en GradeLevel.name/order, NOT NULL en campos obligatorios, FK con onDelete correcto (Cascade en Institution.admin, SetNull en User.institutionId).
- **Lógica de la cuota Free:** índice (userId, viewedAt) en AnnotationView; verificación de que el conteo del mes no incluye vistas duplicadas de la misma anotación.
- **Performance de queries:** explain analyze en queries de catálogo con filtros; no se aceptan seq scan sobre Book con >10k filas.

3.5 Capa 4 - BD No Relacional (MongoDB + Mongoose)

- **Pruebas con Mongo real:** contenedor MongoDB de docker-compose, o mongodb-memory-server para CI rápido.
- **Schema:** validación de SupplementaryMaterial (bookId, pageStart, pageEnd, contenido Markdown, tipo).
- **Limpieza:** deleteMany sobre la colección entre tests.
- **Consultas:** obtener material por bookId y por rango de páginas ($\text{pageStart} \leq p \leq \text{pageEnd}$) con índice compuesto.

3.6 Capa transversal - Seguridad (OWASP ZAP + OWASP Top 10)

- **A01 Broken Access Control:** un Docente no puede acceder a /editor ni a /institution; un Editor no puede editar perfiles de miembros de instituciones de las que no es admin.
- **A02 Cryptographic Failures:** contraseñas hasheadas con bcrypt; JWT con expiración (JWT_EXPIRES_IN=7d) y firmados con JWT_SECRET fuera del repo.
- **A03 Injection:** Prisma usa prepared statements por defecto; los queries en Mongoose se construyen con objetos tipados; Zod valida todos los inputs antes del controller.
- **A05 Security Misconfiguration:** CORS configurado solo al origen del front; headers de seguridad con helmet; mensajes de error sin stack trace en producción.
- **A07 Authentication Failures:** rate-limit en /auth/login; JWT con algoritmo seguro; tokens revocables al cambiar contraseña.

4. Tipos de prueba a ejecutar y justificación

El equipo ejecuta cuatro tipos de prueba principales, cada uno con su herramienta y propósito. Esta decisión es deliberada: las cuatro herramientas cubren el 100% del modelo de calidad acordado con el profesor para esta entrega.

Tipo	Herramienta	Por qué se ejecuta	Cuándo se ejecuta
Unitarias	Jest + ts-jest + Supertest	Detectar bugs de lógica de forma temprana y barata; documentar el comportamiento esperado; soportar refactorings sin miedo.	Local (pre-commit) y en cada Pull Request en CI.
End-to-End	Cypress	Validar flujos críticos completos desde el punto de vista del usuario real, en un navegador real, contra la API y BD reales.	En cada PR contra staging y antes de cada release.
Performance	Apache JMeter	Asegurar que el sistema soporta la carga esperada y degrada de forma controlada. Detectar memory leaks y cuellos de botella.	Antes de cada release mayor y mensualmente en staging.
Seguridad	OWASP ZAP + checklist OWASP Top 10	Detectar vulnerabilidades comunes antes de exponer la plataforma a usuarios reales.	Baseline scan en cada PR; active scan completo antes de cada release.

4.1 Tipos de pruebas de rendimiento con JMeter

JMeter ejecutará cuatro perfiles de carga sobre los endpoints críticos identificados en la sección 2.3:

- **Load Test:** 500 usuarios concurrentes durante 10 minutos con ramp-up de 5 minutos. Objetivo: $p95 \leq 800$ ms sobre /api/auth/login, /api/books, /api/annotations/:id/reveal y /api/catalog.
- **Stress Test:** incremento progresivo desde 100 hasta 2000 usuarios hasta encontrar el punto de quiebre. Objetivo: identificar el límite del sistema y validar que la recuperación es automática al bajar la carga.
- **Spike Test:** pico repentino de 10x el tráfico normal (5000 usuarios) por 1 minuto. Objetivo: el sistema no debe caer; puede degradar respuestas. Verificar que no haya errores 5xx >1% durante el pico.
- **Soak Test (Endurance):** carga normal sostenida (300 usuarios) por 2 horas. Objetivo: detectar memory leaks en Express, fugas de conexiones a Prisma o crecimiento descontrolado del pool de Mongoose.

4.2 Tipos de pruebas de seguridad con OWASP ZAP

- **Baseline Scan:** análisis pasivo, no intrusivo, ejecutado en CI en cada PR contra staging. Tiempo objetivo: <5 minutos.
- **Active Scan:** análisis activo (intenta explotar) ejecutado antes de cada release contra staging, fuera de horario pico. Tiempo objetivo: <60 minutos.
- **Checklist manual OWASP Top 10:** revisión guiada de A01, A02, A03, A05 y A07 (las relevantes para LibroClaro) por el Security Engineer antes de cada release mayor.

5. Criterios de entrada

Se considera que un componente, sprint o release está listo para iniciar la fase formal de pruebas SOLO si se cumplen TODOS los siguientes criterios. Son medibles y verificables; no son opiniones.

5.1 Criterios de entrada por nivel

Nivel	Criterios de entrada (TODOS deben cumplirse)
Unitarias (Jest)	Código compilado sin errores de TypeScript en modo strict. ESLint pasa sin errores. Funciones objetivo implementadas con su firma final. Mocks de Prisma y Mongoose configurados.
E2E (Cypress)	Build de la API y de la Web desplegado en staging y accesible. docker-compose up funciona localmente. Migraciones (prisma migrate deploy) aplicadas. Seed cargado (editor@libroclaro.test / editor1234).
Performance (JMeter)	Build candidato desplegado en staging con configuración idéntica a producción. Datos de prueba con volumen representativo (≥1000 libros, ≥10k anotaciones, ≥100 usuarios). Logging y métricas activos.
Seguridad (OWASP ZAP)	Staging accesible desde la máquina del Security Engineer. Usuario de prueba con cada uno de los tres roles (DOCENTE, ADMIN_INSTITUCION, EDITOR) disponible. CORS_ORIGIN apuntando al dominio de staging del front.
Release a producción	Todos los niveles anteriores pasan. Plan de rollback documentado. Imagen Docker luisfidev/libroclaro:latest publicada y verificada.

6. Criterios de salida

Se considera que la fase de pruebas ha sido completada satisfactoriamente cuando TODOS los siguientes criterios se cumplen simultáneamente. Si alguno falla, la release queda bloqueada hasta su resolución.

6.1 Criterios numéricos

Métrica	Umbral mínimo de salida
Cobertura global de pruebas unitarias (Jest)	≥ 75%

Métrica	Umbral mínimo de salida
Cobertura de la capa de dominio (services)	≥ 90%
Cobertura de la capa de aplicación (controllers + middleware)	≥ 80%
Cobertura de la capa de presentación (componentes React críticos)	≥ 70%
Cobertura de la capa de infraestructura (Prisma/Mongoose wrappers)	≥ 60%
Casos de prueba ejecutados (Cypress + manuales)	100% de los P0 y P1; ≥ 95% del total
Tasa de éxito de los casos de prueba	≥ 98%
Defectos abiertos de severidad Critical o High	0
Defectos abiertos de severidad Medium	≤ 5
p95 de latencia en endpoints P0 (JMeter Load)	≤ 800 ms
Error rate en JMeter Load Test	≤ 0.5%
Vulnerabilidades High/Critical en OWASP ZAP	0
Errores de TypeScript / ESLint	0

6.2 Criterios cualitativos

- Todos los casos de prueba P0 ejecutados al menos una vez en staging con Cypress.
- Sign-off del Product Owner sobre los flujos críticos (login, lector, cuota Free, suscripciones, editor).
- Checklist manual de OWASP Top 10 firmado por el Security Engineer (Mariano Jara).
- Documentación de release notes y plan de rollback firmados.

7. Criterios de suspensión (y reanudación)

Cuando ocurren ciertas condiciones, las actividades de prueba deben SUSPENDERSE para evitar desperdiciar esfuerzo o reportar defectos espurios derivados de un ambiente roto.

7.1 Condiciones que activan suspensión

- **Build de staging caído:** si el ambiente no responde por >15 minutos, se suspende la ejecución de Cypress y JMeter hasta que se restablezca.
- **Bases de datos corruptas o con datos inconsistentes:** si la BD de staging queda en estado inconsistente (por ejemplo, una migración a medias), se suspenden todas las pruebas hasta restaurar seeds limpios con npm run seed.

- **Fallo masivo en Cypress (>25% de tests fallando):** probablemente hay un defecto sistémico de configuración o build. Se suspende y se hace análisis de causa raíz antes de continuar.
- **Defecto bloqueante (Blocker) abierto:** si un defecto impide ejecutar los demás casos del módulo, se suspenden las pruebas de ese módulo hasta su corrección.
- **OWASP ZAP detecta vulnerabilidad Critical activa:** se suspende la fase de pruebas hasta que el equipo de desarrollo aplique un patch.
- **JMeter satura el ambiente compartido:** si una corrida de JMeter degrada el ambiente de staging para otros equipos, se suspende la corrida y se coordina una ventana exclusiva.
- **Cambio mayor de scope a mitad del sprint:** se suspende la ejecución y se reevalúan los casos de prueba afectados.

7.2 Criterios de reanudación

Las pruebas se reanudan cuando la causa de suspensión ha sido resuelta, verificada por al menos dos miembros del equipo (un Dev y un QA), y comunicada al QA Lead (Stella Casillas). Se actualiza el log de incidencias del Test Plan y se vuelve a ejecutar el subconjunto afectado desde cero.

8. Roles y responsabilidades del equipo de QA

El equipo de QA de LibroClaro está conformado por cuatro integrantes, cada uno con un rol principal alineado a una de las cuatro herramientas/áreas de prueba acordadas. Todos los integrantes contribuyen además a las pruebas unitarias del código que tocan.

Integrante	Rol principal	Responsabilidades
Stella Casillas	QA Lead	Elabora y mantiene el Test Plan; coordina al equipo; aprueba criterios de entrada/salida/suspensión; reporta el estado de calidad a stakeholders; decide priorización de defectos junto al Product Owner; mantiene el dashboard de métricas y firma el sign-off final de cada release.
Luis Fernando Maldonado	QA Automation Engineer (Jest + Cypress)	Implementa y mantiene la suite de pruebas unitarias en Jest (controllers, services, middleware, componentes React); implementa y mantiene la suite E2E en Cypress; mantiene los pipelines de CI con GitHub Actions; instrumenta cobertura y reportes.
Mariano Jara	Performance & Security Engineer (JMeter + OWASP ZAP)	Diseña y ejecuta los planes de Load, Stress, Spike y Soak con JMeter; instrumenta endpoints críticos; analiza resultados y propone mejoras de rendimiento (índices Prisma, ajustes de pool,

Integrante	Rol principal	Responsabilidades
		caché); mantiene los reportes de tendencia. Ejecuta los escaneos Baseline y Active de OWASP ZAP; verifica el checklist OWASP Top 10 para LibroClaro (A01, A02, A03, A05, A07); coordina la corrección de findings con desarrollo; firma el reporte de seguridad antes de cada release.
Carlos Morrison	DevOps Engineer (Azure Application Insights)	Configura el logger en el servidor de API conectado a Azure Application Insights; define los logpoints en el código del servidor; monitorea los logs y la estabilidad de la infraestructura en Azure.

8.1 Responsabilidades compartidas

- Los cuatro integrantes participan en la sesión de Bug Bash al cierre de cada sprint (90 minutos).
- Los cuatro integrantes reportan defectos en Jira con el formato definido en la sección 11.
- Los cuatro integrantes mantienen actualizada la documentación de QA en /docs/qa/.
- Cualquier integrante puede solicitar la suspensión de pruebas; la reanudación la autoriza el QA Lead.

8.2 Matriz RACI resumida

Actividad	Mariano	Carlos	Stella	Luis
Test Plan y reporte de estado	C	C	R/A	C
Casos de prueba (manuales)	R	I	A	R
Suite Jest (unitarias)	C	C	C	R/A
Suite Cypress (E2E)	C	C	C	R/A
Plan JMeter (performance)	R/A	C	C	C
Escaneos OWASP ZAP	R/A	C	C	C
Triage de defectos	R/A	R	R	R
Sign-off de release	R/A	R	R	R

Leyenda: R = Responsable de ejecutar · A = Accountable (rinde cuentas) · C = Consultado.

9. Entornos de prueba

El proyecto cuenta con tres entornos perfectamente diferenciados. La configuración de cada uno está documentada como variables de entorno (.env.example) y reproducible vía docker-compose.

Entorno	Propósito	Datos	Disponibilidad	Quién despliega
Local	Desarrollo individual y ejecución de unitarias	Postgres + Mongo en Docker; seed (editor@libroclaro.test)	On-demand	Cada Dev / QA
CI (GitHub Actions)	Ejecución automatizada de Jest, Cypress y ZAP Baseline en cada PR	Postgres + Mongo efímeros; seed cargado en cada run	Por PR	GitHub Actions
Staging (DigitalOcean)	Validación E2E completa, JMeter, ZAP Active y UAT	Datos de prueba con volumen representativo; reset semanal	24/7	Pipeline auto desde rama main
Producción (DigitalOcean)	Tráfico real de usuarios	Datos reales; no se ejecutan pruebas destructivas	24/7 con SLA 99.5%	Pipeline manual con aprobación

9.1 Diferencias clave entre entornos

- **URLs:** local=http://localhost:5173 (web) / :4000/api; staging=https://staging.libroclaro.app; prod=https://libroclaro-kum56.ondigitalocean.app/.
- **BD:** local y CI usan Postgres/Mongo en Docker; staging usa instancias gestionadas por DigitalOcean con backups diarios; producción usa instancias gestionadas con backups + réplica.
- **Storage de PDFs:** local usa STORAGE_DIR=./storage; staging y prod usan volúmenes persistentes.
- **CORS_ORIGIN:** local=http://localhost:5173; staging y prod=dominio del front correspondiente.
- **JWT_EXPIRES_IN:** local=7d; staging=1d (para forzar pruebas de expiración); prod=7d.
- **Logging:** local=stdout; staging y prod a DigitalOcean Logs.

10. Herramientas utilizadas y justificación

De forma deliberada, el equipo limita el toolkit de calidad a cuatro herramientas: una por cada tipo de prueba. Esto reduce la curva de aprendizaje, simplifica el mantenimiento y permite enfocar el esfuerzo. Cada herramienta cubre las cuatro capas del sistema cuando aplica.

Herramienta	Tipo de prueba	Justificación de la elección
Jest + ts-jest + Supertest	Unitarias e integración HTTP	Jest es el estándar de facto en JavaScript/TypeScript y se integra de forma nativa con ts-jest para TypeScript. Supertest permite probar la app Express sin abrir puertos. Combinación bien documentada y soportada por la comunidad.
Cypress	End-to-End (web)	Cypress ofrece DX excepcional para SPAs como React + Vite: time-travel debugging, auto-waits, video y screenshots de cada corrida, ejecución headless en CI. El equipo ya tiene experiencia previa con la herramienta.
Apache JMeter	Performance (Load, Stress, Spike, Soak)	JMeter es la herramienta de facto para pruebas de rendimiento de APIs REST. Permite definir planes complejos vía UI o JMX y ejecutarlos headless en CI. Genera reportes HTML detallados y se integra con Grafana para tendencias.
OWASP ZAP	Seguridad (DAST)	Open source, mantenido por la OWASP Foundation. Permite ejecutar baseline scan en CI (rápido, pasivo) y active scan completo antes de releases. Cubre directamente los puntos del OWASP Top 10.

10.1 Herramientas auxiliares (no de prueba)

- Jira: gestión de defectos y casos de prueba (proyecto LIBROCLARO-QA).
- GitHub Actions: orquestación de CI/CD.
- Docker Compose: ya provisto por el repositorio para levantar el stack completo.
- Prisma Studio (npm run prisma:studio): inspección manual de la BD relacional durante el testing exploratorio.

11. Gestión de defectos

11.1 Flujo del defecto

El ciclo de vida de un defecto sigue la siguiente máquina de estados:

Open → In Progress → Fixed → Verified → Closed

Estados auxiliares: Reopened (cuando un fix falla en la verificación) y Won't Fix (cuando se decide no corregir, con justificación documentada y aprobación del Product Owner).

11.2 Severidad

Severidad	Definición
Critical	El sistema no funciona, hay pérdida o exposición de datos. No existe workaround. Bloquea operación normal. Ejemplo: cualquier usuario autenticado puede leer anotaciones de otros sin descontar cuota.
High	Funcionalidad principal afectada. Workaround complejo. Impacta a la mayoría de usuarios. Ejemplo: el lector PDF no carga libros >50 MB.
Medium	Funcionalidad secundaria afectada o workaround fácil. Impacto a un subconjunto. Ejemplo: filtro por ciclo escolar no preserva el estado al recargar.
Low	Detalle cosmético o de UX menor. No impide el uso. Ejemplo: el tooltip de anotación no respeta el theme oscuro.

11.3 Prioridad

Prioridad	Definición
Blocker	Debe corregirse antes de cualquier otra tarea. Detiene la release.
High	Debe corregirse en el sprint actual.
Medium	Debe corregirse en los próximos dos sprints.
Low	Backlog; se prioriza junto al resto de los items.

11.4 SLAs de respuesta

Severidad	Tiempo máximo de triaje	Tiempo máximo de resolución
Critical	1 hora	8 horas hábiles
High	4 horas	3 días hábiles
Medium	1 día hábil	1 sprint (2 semanas)
Low	1 sprint	Backlog priorizado

11.5 Herramienta y trazabilidad

Los defectos se gestionan en Jira (proyecto LIBROCLARO-QA). Cada defecto debe vincularse a: el caso de prueba que lo detectó (ej. TC-API-014), la historia de usuario originaria (ej. LCL-203) y, una vez corregido, al commit/PR que lo resuelve. Esto garantiza trazabilidad bidireccional Requisito ↔ Caso de prueba ↔ Defecto ↔ Fix.

12. Métricas de calidad objetivo

Las métricas se calculan automáticamente al cierre de cada sprint y se publican en el reporte de calidad que el QA Lead (Stella Casillas) envía a los stakeholders cada lunes.

12.1 Métricas de cobertura (Jest)

Capa	Cobertura mínima
Capa de dominio (services: auth, quota, pdf, markdownPdf, storage)	90%
Capa de aplicación (controllers + middleware)	80%
Capa de presentación (componentes React críticos)	70%
Capa de infraestructura (Prisma/Mongoose wrappers)	60%
Cobertura global del proyecto (API + Web)	75%

12.2 Métricas de ejecución (Cypress)

- **Test Pass Rate:** porcentaje de casos que pasaron sobre los ejecutados. Objetivo $\geq 98\%$.
- **Tasa de automatización de casos P0/P1:** $\geq 80\%$ en Cypress.
- **Tiempo de ejecución del pipeline completo de Cypress:** ≤ 20 minutos en CI.
- **Defect Leakage:** defectos encontrados en producción / defectos totales. Objetivo $\leq 5\%$.

12.3 Métricas de rendimiento (JMeter)

- **p50 / p95 / p99 de latencia por endpoint P0:** $p95 \leq 800$ ms; $p99 \leq 2$ s.
- **Error rate en Load Test:** $\leq 0.5\%$.
- **Throughput sostenido:** ≥ 500 RPS sin degradación.
- **Memoria del proceso de la API tras Soak Test de 2h:** sin crecimiento $>10\%$ respecto a baseline (sin memory leaks).

12.4 Métricas de seguridad (OWASP ZAP)

- **Vulnerabilidades High/Critical:** 0 antes de release.
- **Vulnerabilidades Medium:** ≤ 3 con plan de mitigación documentado.
- **Cobertura de los 10 puntos de OWASP Top 10:** 100% revisados manualmente antes de release mayor.

13. Riesgos identificados y plan de mitigación

Riesgo	Impacto	Mitigación
Subida de PDFs grandes (>50 MB) bloquea el event loop de Express durante la generación de portada con pdf2pic/sharp	Alto	JMeter ejecuta Soak Test con subidas concurrentes; mover la generación a un worker (Bull/BullMQ) si se detecta bloqueo; límite de tamaño documentado en Multer.

Riesgo	Impacto	Mitigación
Cuota Free (20/mes) calculada incorrectamente al concurrir varias vistas simultáneas del mismo usuario	Alto	Pruebas unitarias y E2E con concurrencia (cy.task con múltiples requests); índice (userId, viewedAt) revisado; transacciones atómicas en quota.service.
Endpoint /api/annotations/:id/reveal expuesto sin autenticar permite enumerar anotaciones	Crítico	OWASP ZAP Active scan; verificación manual de A01; pruebas unitarias del middleware auth.
JWT_SECRET filtrado en el repo o en logs	Crítico	Revisión por PR (no se acepta JWT_SECRET en commits); secret manager en staging y prod; logger con allowlist de campos.
Migraciones de Prisma fallan en producción tras release	Alto	Pruebas de migración up + down en CI; backup automático previo al deploy; plan de rollback documentado.
Mongo en producción sin autenticación	Crítico	Variables MONGO_URL con credenciales obligatorias; ZAP verifica que el puerto no esté expuesto; checklist OWASP A05.
Inyección NoSQL en filtros de SupplementaryMaterial	Alto	Validación con Zod antes de pasar al modelo Mongoose; tests unitarios con payloads maliciosos (\$ne, \$gt, etc.).
Cypress flaky por carga inicial del PDF en el lector	Medio	Esperas explícitas en data-testid del lector cuando termina de cargar; retry de 2 intentos; cuarentena automática si 3/5 fallos.
Cambio de roles (DOCENTE → EDITOR) permite escalamiento de privilegios	Crítico	Pruebas unitarias del middleware de rol; ZAP Active scan; verificación manual A01.
Descarga del PDF anotado consume mucha memoria con libros >300 páginas	Alto	JMeter ejecuta Stress sobre /books/:id/annotated; pdf-lib procesado por streams cuando sea posible; límite de páginas documentado.
Docentes institucionales pueden editar su propio perfil pese al bloqueo	Alto	Tests unitarios del middleware de perfil; E2E con usuario institucional que intenta editar; verificación A01.
Pérdida de un integrante del equipo de QA	Medio	Documentación al día en /docs/qa/; pair-testing entre integrantes; runbooks de ejecución de cada suite.

Referencias

- Repositorio del proyecto: /docs/qa/
- OWASP Top 10: <https://owasp.org/Top10/>
- Apache JMeter: <https://jmeter.apache.org/>

- Cypress: <https://www.cypress.io/>
- Jest: <https://jestjs.io/>
- OWASP ZAP: <https://www.zaproxy.org/>
- LibroClaro en producción: <https://libroclaro-kum56.ondigitalocean.app/>